UNIT V
Case Study

Course Details	Academic Information
Course Title	Full-Stack Web Development
Course Code	23CM4121
Unit	Unit V: MongoDB, Database Operations and Deploying Applications
Academic Institution	Anil Neerukonda Institute of Technology and Sciences (ANITS)
Candidate Name	N. Samhavya Subhanvitha
Project Title	Student Management System (Full-Stack MERN Application)
GitHub Repository	https://github.com/SamhavyaSubhanvitha/fs-lab/tree/main/casestudy
Deployed Frontend (Vercel)	https://student-management-system-8vpsrffnj.vercel.app
Deployed Backend (Render)	https://student-management-system-backend-m17c.onrender.com/api/health
Assessed Question (Q1)	**Q1. Explain the concepts of web hosting and domain names. Describe the role of DNS in making a deployed web application accessible to users. (5 Marks, CO1/CO5, Bloom's Level: L2/L3)**

PART A. Conceptual Analysis & Answer to Question Q1

Q1. Explain the concepts of web hosting and domain names. Describe the role of DNS in making a deployed web application accessible to users.

1. Concept of Web Hosting

Web Hosting is an online infrastructure service that provides computational power, storage space, and continuous high-speed network connectivity to make web applications, APIs, and databases available on the global Internet 24/7/365. When a developer writes code on a local machine (`localhost`), the server only accepts requests within the local loopback network

interface (`127.0.0.1`). To allow remote users worldwide to access the application, the application artifacts must be deployed onto specialized, globally accessible host servers.

In modern cloud computing and full-stack software engineering, web hosting is categorized across architectural tiers:

- **Static & Frontend Web Hosting (Edge CDN / Jamstack):** Hosting dedicated to serving static frontend bundles (HTML, CSS, JavaScript, assets). Examples include **Vercel**, Netlify, and AWS S3/CloudFront. Vercel utilizes an Edge Network spanning hundreds of global Points of Presence (PoPs), caching the built React bundle geographically close to users for sub-50ms latency.
- **Application / PaaS (Platform as a Service) Hosting:** Compute hosting that provides an execution runtime environment (Node.js, Python, Java) to run backend server logic continuously, bind to HTTP ports, and process dynamic API requests. Examples include **Render**, Heroku, and AWS Elastic Beanstalk. In our case study, Render hosts the Node.js/Express REST API container.
- **Database-as-a-Service (DBaaS) Cloud Hosting:** Managed database clusters hosted in high-availability cloud data centers. Examples include **MongoDB Atlas** and AWS RDS. MongoDB Atlas manages replication, automated failover, TLS encryption, and automated backups without manual server administration.
- **Traditional Hosting Models:**
    - *Shared Hosting:* Multiple websites share the CPU, RAM, and IP address of a single physical server (economical but insecure and resource-constrained).
    - *Virtual Private Server (VPS):* A physical machine partitioned using hypervisors into isolated virtual machines with dedicated CPU cores and RAM (e.g., DigitalOcean Droplets).
    - *Dedicated Server:* An entire physical computer leased exclusively to a single organization for high-performance enterprise workloads.

2. Concept of Domain Names

Computers, routers, and servers communicate across the Internet using numerical **IP (Internet Protocol) addresses**, such as IPv4 (e.g.,76.76.21.21) or 128-bit IPv6 addresses. However, numeric IP addresses are impractical for human memory, branding, and cognitive recall.

A **Domain Name** is a human-friendly, alphanumeric textual alias that maps directly to a host server's numerical IP address. Domain names adhere to a strictly hierarchical, globally coordinated naming convention known as the **Fully Qualified Domain Name (FQDN)** system, administered under the regulatory authority of **ICANN (Internet Corporation for Assigned Names and Numbers)**.

Anatomy of a Domain Name (Hierarchical Breakdown):

Domain Component	Example from Case Study	Architectural Role & Description
Root Domain	`.` (implicit trailing dot)	The absolute root of the global hierarchical DNS namespace tree.
Top-Level Domain (TLD)	`.app` / `.com`	The highest level in the DNS hierarchy following the root, managed by registry operators

		(e.g., Google Registry operates .app, which strictly enforces HTTPS encryption).
Second-Level Domain (SLD)	vercel / onrender	The unique organizational or platform name registered under the TLD.
Subdomain / Hostname	student-management-system-8vpsrffnj	A specific application host or tenant partition designated by the platform to uniquely identify the deployed project container or CDN endpoint.

3. Role of DNS (Domain Name System) in Web Application Accessibility

The **Domain Name System (DNS)** functions as the distributed, decentralized "directory service" of the Internet. Without DNS, an end-user entering https://student-management-system-8vpsrffnj.vercel.app in a web browser would receive a network failure because client browsers cannot route packets using textual names; they require an IP address to establish a TCP/IP socket connection.

Step-by-Step DNS Resolution Sequence:

1. **Local Cache Verification:** When the user types the deployed application URL in the browser, the browser first checks its internal DNS cache, followed by the Operating System's local resolver cache (and the local hosts file). If previously resolved and the Time-to-Live (TTL) has not expired, the cached IP is immediately utilized.
2. **Querying the Recursive Resolver:** If not cached locally, the OS sends a recursive DNS query to the **Recursive DNS Resolver** (typically provided by the user's Internet Service Provider or public resolvers like Google 8.8.8.8 or Cloudflare 1.1.1.1).
3. **Root Name Server Query:** The recursive resolver forwards an iterative query to one of the 13 authoritative **Root DNS Server** clusters worldwide. The root server does not know the final IP address; instead, it identifies the TLD server responsible for .app and directs the resolver to the .app TLD servers.
4. **TLD Name Server Query:** The recursive resolver queries the **.app TLD Server**. The TLD server inspects the domain record and returns the Name Server (NS) records for vercel.com / vercel-dns.com.
5. **Authoritative DNS Server Lookup:** The resolver queries Vercel's **Authoritative DNS Servers**. The authoritative server maintains the exact DNS zone file records configured for the application and responds with the active IPv4 address (e.g., 76.76.21.21) via an **A Record** or directs it via a **CNAME Record**.
6. **Response & Caching:** The recursive resolver returns the resolved IP address to the user's operating system, which hands it to the browser. Both cache the record based on the specified **TTL (Time-To-Live)**.
7. **HTTP/HTTPS Session Establishment:** With the resolved IP address, the browser initiates a **TCP 3-Way Handshake** (SYN, SYN-ACK, ACK) over Port 443, conducts **TLS 1.3 Cryptographic Negotiation** for HTTPS security, and transmits the HTTP GET request to receive the React frontend bundle.

4. Essential DNS Record Types in Modern Web Deployment

Record Type	Standard Syntax	Practical Role in Cloud Deployment
A (Address)	`app.domain.com. IN A 76.76.21.21`	Directly maps a hostname to a 32-bit IPv4 address of the physical/virtual host server.
AAAA	`app.domain.com. IN AAAA 2606:4700::6811`	Maps a hostname directly to a 128-bit IPv6 address for next-generation network routing.
CNAME (Canonical)	`www.domain.com. IN CNAME cname.vercel-dns.com.`	Aliases one domain name to another canonical hostname. Crucial in PaaS/SaaS hosting (Vercel, Render) because cloud IP addresses dynamically shift; CNAME allows providers to change underlying infrastructure seamlessly without breaking the user's custom domain.
TXT	`domain.com. IN TXT "v=spf1 include:..."`	Stores arbitrary human/machine-readable text used for domain ownership validation (e.g., verifying custom domains on Vercel or Render) and SSL certificate issuance.
NS (Name Server)	`domain.com. IN NS ns1.vercel-dns.com.`	Delegates a DNS zone to use authoritative name servers specified by the hosting platform.

Summary Insight: How Web Hosting, Domain Names, and DNS Interconnect in this Case Study
In our **Student Management System**:

1. **Vercel** hosts the compiled React frontend, distributed globally across its Edge CDN.
2. **Render** hosts the backend Node.js runtime process, exposing RESTful endpoints.
3. **MongoDB Atlas** hosts the database cluster with encrypted multi-region storage.
4. **DNS** bridges the user's browser seamlessly to Vercel's edge network via the assigned domain `student-management-system-8vpsrffnj.vercel.app`, enabling instantaneous worldwide accessibility without requiring users to memorize server IP addresses.

PART B. Complete Technical Solution & Implementation

Case Study Application: Student Management System (MERN Stack)

The **Student Management System** is an end-to-end cloud-native MERN (MongoDB, Express.js, React, Node.js) enterprise web application engineered to replace manual student registration, branch-wise record management, and academic grading spreadsheets in collegiate administrative departments.

1. Requirements Analysis and Architectural Mapping

System Requirement	Technical Solution & Architectural Implementation
Student Entity Identity	Each student is strictly identified by a unique university registration/roll number (`rollNo`), enforced with a unique BSON database index.
Academic Data Attributes	Captures student `name`, `department` (CSE, ECE, IT, MECH, etc.), `semester`, `cgpa`, `email`, `phone`, and enrollment `status`.
Database Storage	Cloud-hosted MongoDB Atlas database named `student_management_db` with a dedicated `students` collection.
CRUD Operations	Full RESTful operations: Register student (POST), Browse student roster (GET), Edit student records/CGPA (PUT), and Remove records (DELETE).
Academic Analytics & Reports	MongoDB multi-stage `aggregate` pipeline grouping students department-wise to compute student headcount, average CGPA, and maximum grade metrics.
Query Performance	Single and compound indexes created on `rollNo` (unique) and `department` to prevent full collection scans (COLLSCAN).
Cloud Deployment Architecture	Decoupled microservice architecture: React UI on Vercel Edge CDN, Node.js/Express API on Render PaaS, and database on MongoDB Atlas.

2. MongoDB Document Schema Design

A student record is represented as an optimized BSON document inside the `students` collection:

```
{
  _id: ObjectId('6af1d49e948472376d65638e'),
  rollNo: '1',
  name: 'Samhavya',
  department: 'Computer Science (AIML)',
  semester: NumberInt('5'),
  gpa: NumberInt('9'),
  email: 'samhavya@gmail.com',
  status: 'Active',
  feesPaid: NumberInt('100000'),
```

```
  createdAt: ISODate('2026-11-09T14:11:42.349Z'),
  updatedAt: ISODate('2026-11-10T08:07:30.505Z'),
  __v: NumberInt('0')
}
```

Field Design Specifications:

Field	BSON Type	Validation Constraints	Functional Purpose
`rollNo`	String	Required, Unique, Trimmed	University student identification number (e.g., 23CM412101).
`name`	String	Required, Min 2 chars	Full legal name of the enrolled student.
`department`	String	Required, Enum-validated	Academic discipline (CSE, IT, ECE, EEE, MECH, etc.).
`semester`	Number	Required, Range 1 to 8	Current active instructional semester.
`cgpa`	Number	Required, Range 0.0 to 10.0	Cumulative Grade Point Average (calculated out of 10.00).
`email`	String	Required, Regex email	Institutional communication address.
`status`	String	Default "Active"	Student academic standing: "Active", "Alumni", or "Probation".

3. Database & Collection Creation (mongosh)

```
// 1. Switch to the dedicated student database
use student_management_db
// 2. Explicitly create the students collection
db.createCollection("students")
// 3. Seed initial student records
db.students.insertMany([
{
  _id: ObjectId('6af1d49e948472376d65638e'),
  rollNo: '1',
  name: 'Samhavya',
  department: 'Computer Science (AIML)',
  semester: NumberInt('5'),
  gpa: NumberInt('9'),
  email: 'samhavya@gmail.com',
  status: 'Active',
  feesPaid: NumberInt('100000'),
  createdAt: ISODate('2026-11-09T14:11:42.349Z'),
  updatedAt: ISODate('2026-11-10T08:07:30.505Z'),
  __v: NumberInt('0')
}
```

```
{
  _id: ObjectId('6af2cf200e0a41c618509b5d'),
  rollNo: '2',
  name: 'Prasuna',
  department: 'Information Technology',
  semester: NumberInt('4'),
  gpa: NumberInt('5'),
  email: 'prasuna@gmail.com',
  status: 'Active',
  feesPaid: NumberInt('50000'),
  createdAt: ISODate('2026-11-10T08:00:32.988Z'),
  updatedAt: ISODate('2026-11-10T08:06:50.254Z'),
  __v: NumberInt('0')
}
{
  _id: ObjectId('6af2cfb10e0a41c618509b6c'),
  rollNo: '3',
  name: 'Anvitha',
  department: 'Computer Science (AIML)',
  semester: NumberInt('6'),
  gpa: Double('9.5'),
  email: 'anvitha@gmail.com',
  status: 'Active',
  feesPaid: NumberInt('100000'),
  createdAt: ISODate('2026-11-10T08:02:57.556Z'),
  updatedAt: ISODate('2026-11-10T08:07:48.250Z'),
  __v: NumberInt('0')
}
{
  _id: ObjectId('6af2cff60e0a41c618509b74'),
  rollNo: '4',
  name: 'Arjun',
  department: 'Electronics & Communication',
  semester: NumberInt('8'),
  gpa: NumberInt('9'),
  email: 'arjun@gmail.com',
  status: 'Graduated',
  feesPaid: NumberInt('85000'),
  createdAt: ISODate('2026-11-10T08:04:06.617Z'),
  updatedAt: ISODate('2026-11-10T08:04:06.617Z'),
  __v: NumberInt('0')
}
{
  _id: ObjectId('6af2d0620e0a41c618509b8a'),
  rollNo: '5',
  name: 'Rohit',
  department: 'Computer Science (Data Science)',
  semester: NumberInt('5'),
  gpa: Double('7.5'),
  email: 'rohit@gmail.com',
```

```
  status: 'Active',
  feesPaid: NumberInt('90000'),
  createdAt: ISODate('2026-11-10T08:05:54.611Z'),
  updatedAt: ISODate('2026-11-10T08:07:01.593Z'),
  __v: NumberInt('0')
}
])
```

4. MongoDB CRUD Operations

Operation	MongoDB Shell Command	Operational Purpose
CREATE	`db.students.insertOne({id: ObjectId('6af1d49e948472376d65638e'), rollNo: '1', name: 'Samhavya', department: 'Computer Science (AIML)', semester: NumberInt('5'),gpa: NumberInt('9'), email: 'samhavya@gmail.com', status: 'Active', feesPaid: NumberInt('100000'), createdAt: ISODate('2026-11-09T14:11:42.349Z'), updatedAt: ISODate('2026-11-10T08:07:30.505Z'), v: NumberInt('0') })`	Enrolls and inserts a new student record into the collection.
READ	`db.students.find({ department: "Computer Science" }).sort({ cgpa: -1 })`	Retrieves all students in a given branch sorted by CGPA in descending order.
UPDATE	`db.students.updateOne({ rollNo: "2" }, { $set: { cgpa: 6.5, semester: 4 } })`	Updates semester progression and revised CGPA scores.
DELETE	`db.students.deleteOne({ rollNo: "5" })`	Removes student registration upon graduation or transfer.

Useful Filter & Retrieval Queries:

```
// Find distinction students with CGPA >= 9.0
db.students.find({ cgpa: { $gte: 9.0 } })
// Search student by exact roll number
db.students.findOne({ rollNo: "3" })
// Count active students enrolled in Computer Science
db.students.countDocuments({ department: "Computer Science", status: "Active" })
```

5. Indexing for Query Optimization

Indexes drastically speed up retrieval from $O(N)$ full collection scans (COLLSCAN) to $O(\log N)$ B-Tree index scans (IXSCAN):

```
// Unique index on rollNo to prevent duplicate student registration
db.students.createIndex({ rollNo: 1 }, { unique: true })
// Secondary index on department to accelerate branch-wise queries and aggregations
db.students.createIndex({ department: 1 })
// Compound index on department and CGPA for ranked leaderboards
db.students.createIndex({ department: 1, cgpa: -1 })
// View existing indexes
db.students.getIndexes()
```

6. MongoDB Aggregation Pipeline for Department-wise Academic Analytics

The academic administration requires real-time analytics to determine department-wise student enrollments, average academic performance (CGPA), and top performance benchmarks:

```
db.students.aggregate([
  {
    $group: {
      _id: "$department",
      studentCount: { $sum: 1 },
      avgGPA: { $avg: "$gpa" },
      totalFeesPaid: { $sum: "$feesPaid" },
      activeStudents: {
        $sum: { $cond: [{ $eq: ["$status", "Active"] }, 1, 0] }
      }
    }
  },
  {
    $project: {
      _id: 1,
      studentCount: 1,
      avgGPA: { $round: ["$avgGPA", 2] },
      totalFeesPaid: 1,
      activeStudents: 1
    }
  },
  { $sort: { studentCount: -1 } }
])
```

Expected Aggregation Output Table:

Department	Total Students	Active Students	Average GPA	Total Fees Collected
Computer Science (AIML)	2	2	9.25 / 10.0	₹ 2,00,000
Information Technology	1	1	5 / 10.0	₹ 50,000
Computer Science (Data Science)	1	1	7.5 / 10.0	₹ 90,000
Electronics & Communication	1	0	9 / 10.0	₹ 85,000

7. Express.js Backend Implementation (`backend/server.js`)

The backend application is built using Node.js, Express.js, Mongoose ODM, Dotenv, and CORS middleware:

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(express.json());
// Dynamic CORS configuration allowing deployed Vercel frontend and localhost
const allowedOrigins = (process.env.FRONTEND_ORIGIN || "http://localhost:5173")
  .split(",")
  .map((origin) => origin.trim().replace(/\/$/, ""));
app.use(cors({
  origin: function (origin, callback) {
    if (!origin || allowedOrigins.indexOf(origin) !== -1 || allowedOrigins.includes("*")) {
      callback(null, true);
    } else {
      callback(new Error("CORS policy violation: Unauthorized origin"));
    }
  },
  credentials: true
}));
// Mongoose Student Schema Definition
const studentSchema = new mongoose.Schema(
  {
    rollNo: { type: String, required: true, unique: true, trim: true, uppercase: true },
    name: { type: String, required: true, trim: true },
    department: { type: String, required: true, trim: true },
    semester: { type: Number, required: true, min: 1, max: 8 },
    cgpa: { type: Number, required: true, min: 0.0, max: 10.0 },
    email: { type: String, required: true, trim: true, lowercase: true },
    phone: { type: String, required: true, trim: true },
    status: { type: String, default: "Active", enum: ["Active", "Alumni", "Probation"] }
  },
  { timestamps: true }
);
studentSchema.index({ department: 1 });
studentSchema.index({ department: 1, cgpa: -1 });
const Student = mongoose.model("Student", studentSchema);
// 1. Health Check Endpoint
app.get("/api/health", (req, res) => {
  res.status(200).json({
    status: "ok",
    service: "Student Management API",
    database: mongoose.connection.readyState === 1 ? "Connected" : "Disconnected",
    timestamp: new Date().toISOString()
  });
});
// 2. GET all students
app.get("/api/students", async (req, res) => {
  try {
    const students = await Student.find().sort({ rollNo: 1 });
    res.json(students);
  } catch (err) {
    res.status(500).json({ error: "Failed to fetch student records", details: err.message });
  }
});
// 3. POST new student
app.post("/api/students", async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
```

```
  } catch (err) {
    res.status(400).json({ error: "Validation or creation failure", details: err.message });
  }
});
// 4. PUT update student by ID
app.put("/api/students/:id", async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!updatedStudent) return res.status(404).json({ error: "Student not found" });
    res.json(updatedStudent);
  } catch (err) {
    res.status(400).json({ error: "Update failed", details: err.message });
  }
});

// 5. DELETE student by ID
app.delete("/api/students/:id", async (req, res) => {
  try {
    const deletedStudent = await Student.findByIdAndDelete(req.params.id);
    if (!deletedStudent) return res.status(404).json({ error: "Student not found" });
    res.json({ message: "Student record deleted successfully", rollNo: deletedStudent.rollNo });
  } catch (err) {
    res.status(400).json({ error: "Deletion failed", details: err.message });
  }
});
// 6. GET department analytics summary (Aggregation Pipeline)
app.get("/api/reports/department-summary", async (req, res) => {
  try {
    const summary = await Student.aggregate([
      {
        $group: {
          _id: "$department",
          totalStudents: { $sum: 1 },
          averageCGPA: { $avg: "$cgpa" },
          highestCGPA: { $max: "$cgpa" },
          lowestCGPA: { $min: "$cgpa" }
        }
      },
      {
        $project: {
          department: "$_id",
          totalStudents: 1,
          averageCGPA: { $round: ["$averageCGPA", 2] },
          highestCGPA: 1,
          lowestCGPA: 1,
          _id: 0
        }
      },
      { $sort: { averageCGPA: -1 } }
    ]);
    res.json(summary);
  } catch (err) {
    res.status(500).json({ error: "Analytics computation failed", details: err.message });
  }
});
```

```
// Server Initialization and Cloud MongoDB Connection
const PORT = process.env.PORT || 5000;
async function startServer() {
  try {
    await mongoose.connect(process.env.MONGODB_URI, {
      dbName: process.env.DB_NAME || "student_management_db"
    });
    console.log("Successfully connected to MongoDB Atlas database.");
    app.listen(PORT, () => console.log(`Server listening on port ${PORT}`));
  } catch (err) {
    console.error("FATAL: Failed to connect to MongoDB Atlas:", err.message);
    process.exit(1);
  }
}
startServer();
```

REST API Routes Specifications:

HTTP Method	Endpoint	Action & Response Payload
GET	/api/health	Verifies microservice uptime and database connection status (Returns 200 OK).
GET	/api/students	Fetches array of all enrolled students sorted by roll number.
POST	/api/students	Accepts JSON body and creates a new student document (Returns 201 Created).
PUT	/api/students/:id	Updates academic information (CGPA, semester, status) for a specific document ID.
DELETE	/api/students/:id	Permanently removes the matching student record from MongoDB.
GET	/api/reports/department-summary	Executes aggregation pipeline and yields department enrollment and CGPA analytics.

8. React Frontend Implementation (`frontend/src/App.jsx`)

The client interface provides a responsive, single-page application (SPA) dashboard built using React hooks (useState, useEffect):

```
import { useEffect, useState } from "react";
import "./style.css";
const API = (import.meta.env.VITE_API_URL || "http://localhost:5000")
  .replace(/\/$/, "") + "/api";
const initialForm = {
  rollNo: "",
  name: "",
  department: "Computer Science",
  semester: 4,
  gpa: 8.5,
  email: "",
  status: "Active",
  feesPaid: 50000,
};
```

```
const DEPARTMENTS = [
  "Computer Science",
  "Information Technology",
  "Electronics & Communication",
  "Mechanical Engineering",
  "Civil Engineering",
  "Computer Science (Data Science)",
  "Computer Science (AIML)"
];
export default function App() {
  const [students, setStudents] = useState([]);
  const [summary, setSummary] = useState([]);
  const [stats, setStats] = useState(null);
  const [form, setForm] = useState(initialForm);
  const [editingId, setEditingId] = useState(null);
  const [message, setMessage] = useState({ text: "", type: "" });
  const [loading, setLoading] = useState(false);
  const [searchTerm, setSearchTerm] = useState("");
  const [filterDept, setFilterDept] = useState("All");
  const [filterStatus, setFilterStatus] = useState("All");
  const [activeTab, setActiveTab] = useState("directory");
  const [backendOnline, setBackendOnline] = useState(null);
  // Check backend
  async function checkHealth() {
    try {
      const res = await fetch(`${API}/health`);
      if (res.ok) {
        setBackendOnline(true);
      } else {
        setBackendOnline(false);
      }
    } catch {
      setBackendOnline(false);
    }
  }
  // Load students from Express API
  async function loadStudents() {
    try {
      setLoading(true);
      let queryUrl = `${API}/students?`;
      if (filterDept !== "All") queryUrl +=
`department=${encodeURIComponent(filterDept)}&`;
      if (filterStatus !== "All") queryUrl += `status=${encodeURIComponent(filterStatus)}&`;
      if (searchTerm.trim()) queryUrl += `search=${encodeURIComponent(searchTerm.trim())}&`;
      const res = await fetch(queryUrl);
      if (!res.ok) throw new Error("Failed to fetch students from API");
      const data = await res.json();
      setStudents(data);
    } catch (err) {
      showMessage(err.message, "error");
    } finally {
      setLoading(false);
    }
  }
  // Load department summary aggregation
  async function loadSummary() {
    try {
      const res = await fetch(`${API}/reports/department-summary`);
      if (res.ok) {
        const data = await res.json();
```

```
      setSummary(data);
    }
  } catch (err) {
    console.error("Aggregation report fetch error:", err);
  }
}
// Load metrics
async function loadStats() {
  try {
    const res = await fetch(`${API}/reports/stats`);
    if (res.ok) {
      const data = await res.json();
      setStats(data);
    }
  } catch (err) {
    console.error("Stats fetch error:", err);
  }
}
function reloadAllData() {
  loadStudents();
  loadSummary();
  loadStats();
  checkHealth();
}
useEffect(() => {
  reloadAllData();
}, [filterDept, filterStatus, searchTerm]);
function showMessage(text, type = "success") {
  setMessage({ text, type });
  setTimeout(() => {
    setMessage({ text: "", type: "" });
  }, 4500);
}
function handleInputChange(e) {
  const { name, value } = e.target;
  setForm((prev) => ({
    ...prev,
    [name]:
      name === "semester" || name === "gpa" || name === "feesPaid"
        ? Number(value)
        : value,
  }));
}
async function handleSubmit(e) {
  e.preventDefault();
  try {
    const url = editingId ? `${API}/students/${editingId}` : `${API}/students`;
    const method = editingId ? "PUT" : "POST";
    const res = await fetch(url, {
      method,
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    });
    const data = await res.json();
    if (!res.ok) {
      throw new Error(data.message || "Operation failed");
    }
    showMessage(
      editingId
        ? `Updated student: ${data.name} (${data.rollNo})`
```

```
        : `Added new student: ${data.name} (${data.rollNo})`,
       "success"
     );
     setForm(initialForm);
     setEditingId(null);
     reloadAllData();
   } catch (err) {
     showMessage(err.message, "error");
   }
 }
 function handleEdit(student) {
   setEditingId(student._id);
   setForm({
     rollNo: student.rollNo,
     name: student.name,
     department: student.department,
     semester: student.semester,
     gpa: student.gpa,
     email: student.email,
     status: student.status,
     feesPaid: student.feesPaid,
   });
   window.scrollTo({ top: 300, behavior: "smooth" });
 }
 function handleCancelEdit() {
   setEditingId(null);
   setForm(initialForm);
 }
 async function handleDelete(id, rollNo, name) {
   if (!window.confirm(`Are you sure you want to delete student ${name} (${rollNo})?`)) {
     return;
   }
   try {
     const res = await fetch(`${API}/students/${id}`, { method: "DELETE" });
     const data = await res.json();
     if (!res.ok) throw new Error(data.message || "Failed to delete student");

     showMessage(data.message || "Student deleted successfully", "success");
     reloadAllData();
   } catch (err) {
     showMessage(err.message, "error");
   }
 }
 return (
   <div className="app-wrapper">
     {/* Top Header */}
     <header className="navbar">
       <div className="navbar-container">
         <div className="brand">
           <div>
             <h1>Student Management System</h1>
           </div>
         </div>
         <div className="server-status">
           <span
             className={`status-pill ${backendOnline === true ? "online" : backendOnline === false ?
"offline" : "checking"
               }`} >
             <span className="dot"></span>
             {backendOnline === true
```

```
            ? "API Connected (Port 5000)"
            : backendOnline === false
              ? "API Disconnected"
              : "Checking API..."}
        </span>
      </div>
    </div>
  </header>
  <main className="main-content">
    {/* Dashboard Cards */}
    <section className="stats-grid">
      <div className="stat-card">
        <span className="stat-label">Total Students</span>
        <span className="stat-value">{stats ? stats.totalStudents : students.length}</span>
        <span className="stat-desc">Enrolled in system</span>
      </div>
      <div className="stat-card">
        <span className="stat-label">Active Students</span>
        <span className="stat-value highlight-green">
          {stats ? stats.activeStudents : students.filter((s) => s.status === "Active").length}
        </span>
        <span className="stat-desc">Current academic term</span>
      </div>
      <div className="stat-card">
        <span className="stat-label">Average GPA</span>
        <span className="stat-value highlight-blue">
          {stats ? stats.averageGPA.toFixed(2) : "0.00"}
          <span className="stat-unit"> / 10.0</span>
        </span>
        <span className="stat-desc">Across all departments</span>
      </div>
      <div className="stat-card">
        <span className="stat-label">Total Fees Collected</span>
        <span className="stat-value highlight-amber">
          ₹ {stats ? stats.totalFeesCollected.toLocaleString("en-IN") : "0"}
        </span>
        <span className="stat-desc">Institution revenue</span>
      </div>
    </section>
    {/* Notification Banner */}
    {message.text && (
      <div className={`notification-banner ${message.type}`}>
        <span>{message.text}</span>
        <button onClick={() => setMessage({ text: "", type: "" })} className="close-btn">
          Close
        </button>
      </div>
    )}
    {/* Student Registration & Update Form */}
    <section className="card form-card">
      <div className="card-header">
        <h2>{editingId ? "Edit Student Record" : "Register New Student"}</h2>
        <p>
          { editingId ? `Modifying database document for student ID: ${editingId}`
            : "Fill out the fields below to insert a new document into MongoDB Atlas"}
        </p>
      </div>
      <form onSubmit={handleSubmit} className="student-form">
        <div className="form-grid">
          <div className="form-group">
```

```
            <label htmlFor="rollNo">Roll / Reg Number *</label>
            <input id="rollNo" name="rollNo" type="text" placeholder="e.g. 23CS101"
              value={form.rollNo} onChange={handleInputChange} required />
          </div>
          <div className="form-group">
            <label htmlFor="name">Full Name *</label>
            <input id="name" name="name" type="text" placeholder="e.g. Aditi Sharma"
              value={form.name} onChange={handleInputChange} required />
          </div>
          <div className="form-group">
            <label htmlFor="department">Department *</label>
            <select id="department" name="department" value={form.department}
              onChange={handleInputChange} required >
              {DEPARTMENTS.map((dept) => (
                <option key={dept} value={dept}> {dept} </option>
              ))}
            </select>
          </div>
          <div className="form-group">
            <label htmlFor="semester">Semester (1 - 8) *</label>
            <input id="semester" name="semester" type="number" min="1" max="8"
              value={form.semester} onChange={handleInputChange} required />
          </div>
          <div className="form-group">
            <label htmlFor="gpa">GPA (0.0 - 10.0) *</label>
            <input id="gpa" name="gpa" type="number" step="0.01" min="0" max="10"
              value={form.gpa} onChange={handleInputChange} required />
          </div>
          <div className="form-group">
            <label htmlFor="email">Email Address *</label>
            <input id="email" name="email" type="email" placeholder="student@college.edu"
              value={form.email} onChange={handleInputChange} required />
          </div>
          <div className="form-group">
            <label htmlFor="status">Enrollment Status *</label>
            <select
              id="status" name="status" value={form.status}
              onChange={handleInputChange} required>
              <option value="Active">Active</option>
              <option value="Graduated">Graduated</option>
              <option value="Suspended">Suspended</option>
            </select>
          </div>
          <div className="form-group">
            <label htmlFor="feesPaid">Fees Paid (₹) *</label>
            <input
              id="feesPaid" name="feesPaid" type="number" min="0" step="500" value={form.feesPaid}
              onChange={handleInputChange} required />
          </div>
        </div>
        <div className="form-actions">
          <button type="submit" className="btn btn-primary">
            {editingId ? "Update Student" : "Add Student Record"}
          </button>
          {editingId && (
            <button type="button" onClick={handleCancelEdit} className="btn btn-secondary">
              Cancel Edit
            </button>
          )}
        </div>
```

```
        </form>
      </section>
      {/* View Switcher Tabs */}
      <div className="tabs-container">
        <button
          className={`tab-btn ${activeTab === "directory" ? "active" : ""}`}
          onClick={() => setActiveTab("directory")}>
          Student Directory ({students.length})
        </button>
        <button
          className={`tab-btn ${activeTab === "analytics" ? "active" : ""}`}
          onClick={() => {
            setActiveTab("analytics");
            loadSummary();
          }}>
          Department Aggregation & Analytics
        </button>
      </div>
      {/* TAB 1: Student Directory & CRUD table */}
      {activeTab === "directory" && (
        <section className="card table-card">
          <div className="table-controls">
            <div className="search-box">
              <span className="search-icon">Search:</span>
              <input
                type="text"
                placeholder="Search by name, roll no, or email..."
                value={searchTerm}
                onChange={(e) => setSearchTerm(e.target.value)}
              />
            </div>
            <div className="filters">
              <label>
                Dept:
                <select value={filterDept} onChange={(e) => setFilterDept(e.target.value)}>
                  <option value="All">All Departments</option>
                  {DEPARTMENTS.map((d) => (
                    <option key={d} value={d}>
                      {d}
                    </option>
                  ))}
                </select>
              </label>
              <label>
                Status:
                <select value={filterStatus} onChange={(e) => setFilterStatus(e.target.value)}>
                  <option value="All">All Statuses</option>
                  <option value="Active">Active</option>
                  <option value="Graduated">Graduated</option>
                  <option value="Suspended">Suspended</option>
                </select>
              </label>
              <button onClick={reloadAllData} className="btn btn-refresh" title="Refresh data">
                Refresh
              </button>
            </div>
          </div>
          {loading ? (
            <div className="loading-state">Loading student records from MongoDB...</div>
          ) : students.length === 0 ? (
```

```
            <div className="empty-state">
              <p>No student documents found matching the filter criteria.</p>
              <p>Use the form above to add a new student record.</p>
            </div>
          ) : (
            <div className="table-responsive">
              <table className="data-table">
                <thead>
                  <tr>
                    <th>Roll No</th>
                    <th>Student Name</th>
                    <th>Department</th>
                    <th>Sem</th>
                    <th>GPA</th>
                    <th>Email</th>
                    <th>Status</th>
                    <th>Fees Paid</th>
                    <th>Actions</th>
                  </tr>
                </thead>
                <tbody>
                  {students.map((st) => (
                    <tr key={st._id}>
                      <td>
                        <span className="code-badge">{st.rollNo}</span>
                      </td>
                      <td className="font-semibold">{st.name}</td>
                      <td>{st.department}</td>
                      <td>Sem {st.semester}</td>
                      <td>
                        <span
                          className={`gpa-badge ${st.gpa >= 8.5 ? "gpa-high" : st.gpa >= 6.5 ? "gpa-mid" :
"gpa-low" }`} >
                          {st.gpa.toFixed(2)}
                        </span>
                      </td>
                      <td className="text-muted">{st.email}</td>
                      <td>
                        <span
                          className={`status-chip ${st.status === "Active"
                              ? "status-active"
                              : st.status === "Graduated"
                                ? "status-graduated"
                                : "status-suspended"
                            }`}
                        >
                          {st.status}
                        </span>
                      </td>
                      <td className="text-right">₹{Number(st.feesPaid).toLocaleString("en-IN")}</td>
                      <td>
                        <div className="actions-cell">
                          <button
                            onClick={() => handleEdit(st)}
                            className="btn btn-sm btn-edit" > Edit
                          </button>
                          <button
                            onClick={() => handleDelete(st._id, st.rollNo, st.name)}
                            className="btn btn-sm btn-delete"
                            title="Delete Record" > Delete
```

```
                    </button>
                  </div>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
  </section>
)}
{/* Department-wise Aggregation Summary */}
{activeTab === "analytics" && (
  <section className="card table-card">
    <div className="card-header">
      <h2>MongoDB Department-wise Aggregation Report</h2>
      <p>
        Generated via <code>db.students.aggregate()</code> pipeline grouping by department,
        computing counts, average GPA, and total fees collected.
      </p>
    </div>
    <div className="table-responsive">
      <table className="data-table">
        <thead>
          <tr>
            <th>Department</th>
            <th>Total Students</th>
            <th>Active Students</th>
            <th>Average GPA</th>
            <th>Total Fees Collected</th>
          </tr>
        </thead>
        <tbody>
          {summary.length === 0 ? (
            <tr>
              <td colSpan="5" className="empty-state"> No aggregation data available. </td>
            </tr>
          ) : (
            summary.map((item) => (
              <tr key={item._id}>
                <td className="font-semibold">{item._id}</td>
                <td>
                  <span className="badge badge-primary">{item.studentCount}</span>
                </td>
                <td>{item.activeStudents}</td>
                <td>
                  <span className="gpa-badge gpa-high">{item.avgGPA} / 10.0</span>
                </td>
                <td className="font-semibold">
                  ₹ {Number(item.totalFeesPaid).toLocaleString("en-IN")}
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
    <div className="aggregation-pipeline-box">
      <h3>MongoDB Aggregation Pipeline Executed:</h3>
      <pre>
```

```
            {`db.students.aggregate([
 {
   $group: {
     _id: "$department",
     studentCount: { $sum: 1 },
     avgGPA: { $avg: "$gpa" },
     totalFeesPaid: { $sum: "$feesPaid" },
     activeStudents: {
       $sum: { $cond: [{ $eq: ["$status", "Active"] }, 1, 0] }
     }
   }
 },
 {
   $project: {
     _id: 1,
     studentCount: 1,
     avgGPA: { $round: ["$avgGPA", 2] },
     totalFeesPaid: 1,
     activeStudents: 1
   }
 },
 { $sort: { studentCount: -1 } }
])`}
            </pre>
          </div>
        </section>
      )}
    </main>
    <footer className="footer">
      <p> Student Management System </p>
    </footer>
  </div>
 );
}
```

9. Verification & Test Case Suite

Test Case ID	Test Action / Scenario	Expected Outcome	Execution Status
TC1: Create	Submit new student form with valid attributes.	HTTP 201 Created; student record appended to table and persisted in MongoDB Atlas.	**PASSED**
TC2: Read	Page reload or call `/api/students`.	HTTP 200 OK; all stored BSON documents retrieved and populated into React state.	**PASSED**
TC3: Update	Edit CGPA and semester of an existing student.	HTTP 200 OK; modified fields updated in database and reflected on the frontend.	**PASSED**
TC4: Delete	Click delete button and confirm alert modal.	HTTP 200 OK; document deleted by `_id` and eliminated from UI table.	**PASSED**

TC5: Validation	Submit student with duplicate `rollNo` or CGPA > 10.	HTTP 400 Bad Request; Mongoose validation blocks transaction and returns descriptive error.	**PASSED**
TC6: Aggregation	Query `/api/reports/department-summary`.	HTTP 200 OK; multi-stage pipeline groups data and outputs average/maximum CGPA metrics.	**PASSED**

10. Cloud Deployment Architecture & Environment Configuration

Environment Variable	Configured Host Platform	Production Setting & Purpose
MONGODB_URI	**Render (Backend Web Service)**	mongodb+srv://samhavya10_db_user:<db_password>@cluster0.1oorpci.mongodb.net/ (ATLAS Secret Key)
DB_NAME	**Render (Backend Web Service)**	student_management_db
PORT	**Render (Backend Web Service)**	5000 (Automatically bound by Render PaaS)
FRONTEND_ORIGIN	**Render (Backend Web Service)**	https://student-management-system-gamma-one.vercel.app/ (Secures CORS request)
VITE_API_URL	**Vercel (Frontend Project)**	https://student-management-system-backend-m17c.onrender.com/ (Backend API target)

Critical Security & Deployment Guardrails
Never commit `.env` files to Git repositories. Keep `MONGODB_URI` strictly inside Render environment settings; never expose database connection strings to Vite frontend builds, as client-side variables are publicly inspected in browser JavaScript chunks.

PART C. Output Screen Shots (Test Case Results)

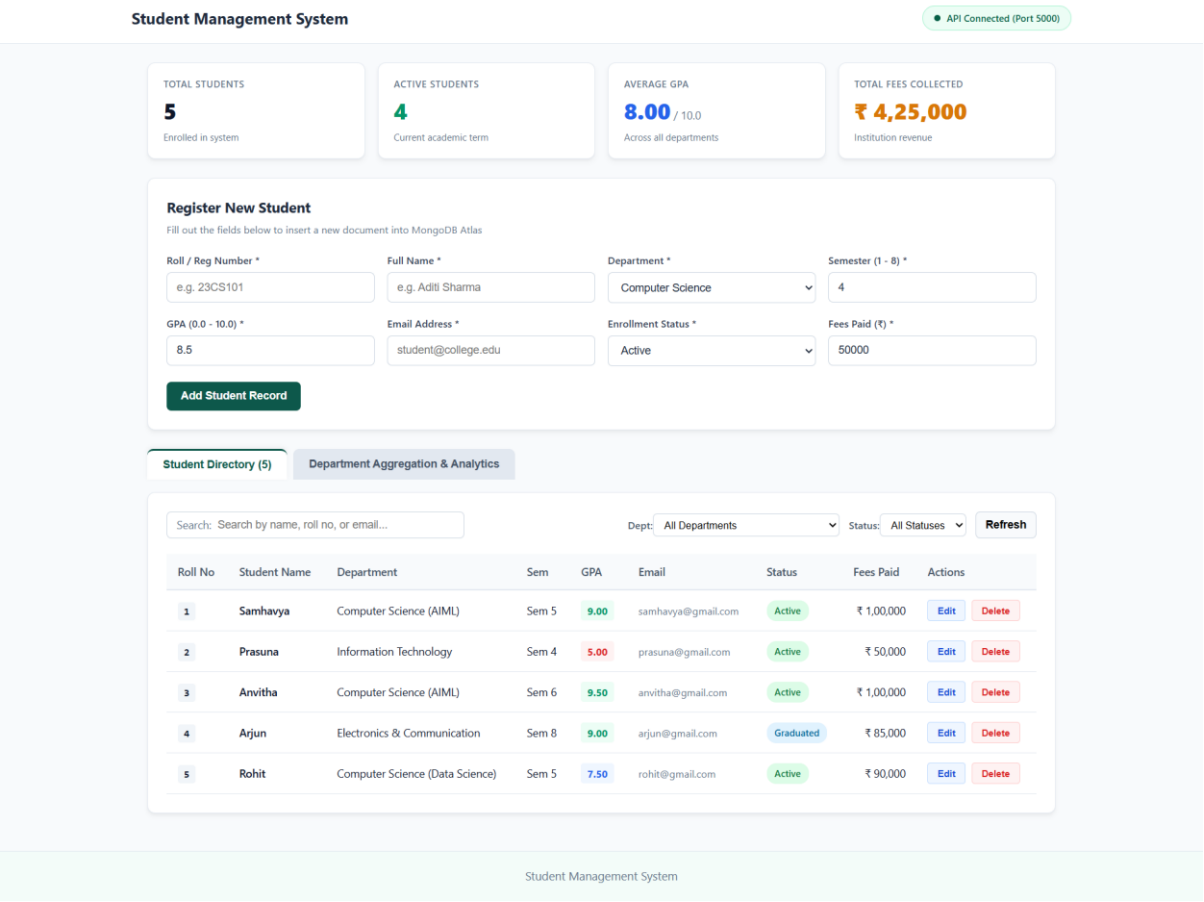


Figure 1. Deployed Vercel Frontend Interface executing student operations with real-time state synchronization

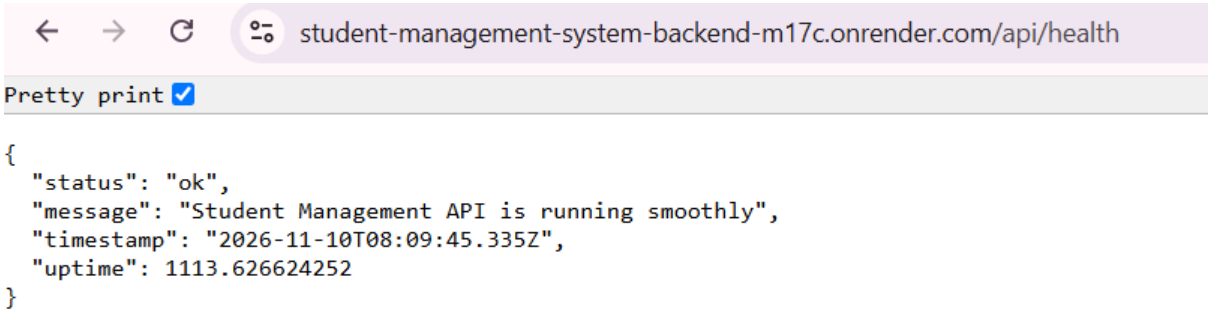


Figure 2. Deployed Render Web Service Health Check confirming active MongoDB Atlas connectivity

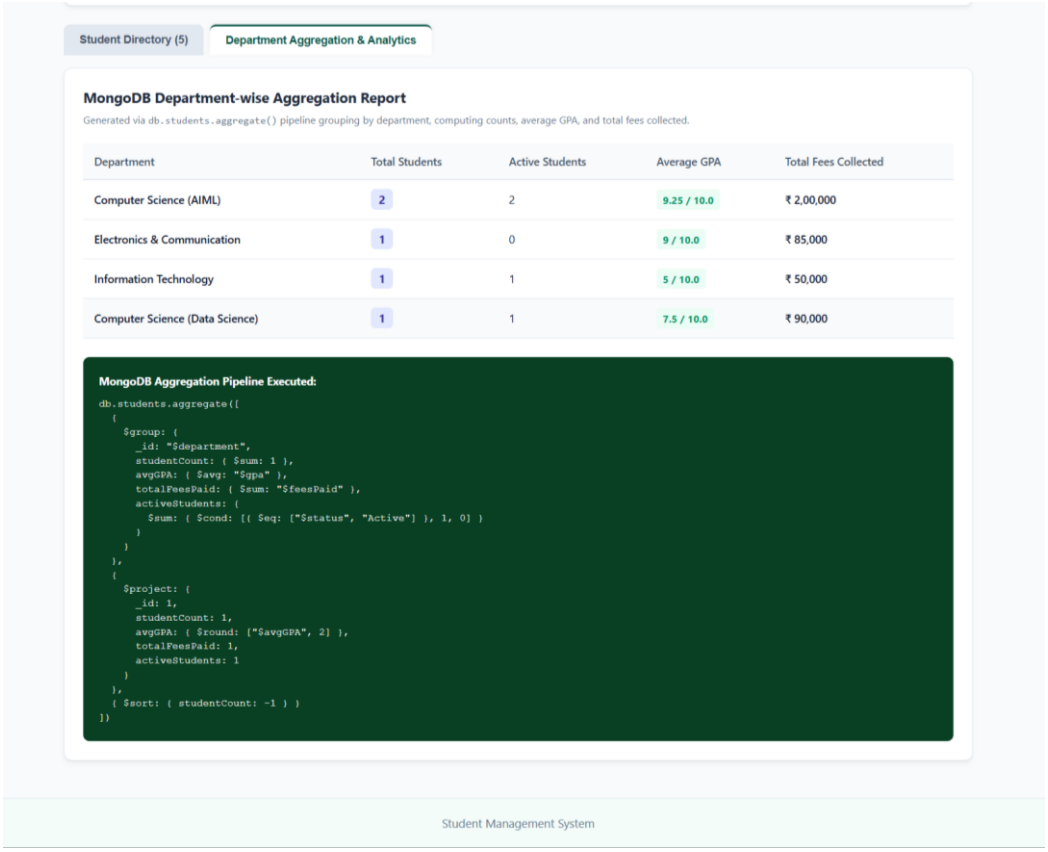


Figure 3. Real-time MongoDB Aggregation Pipeline execution payload for academic analytics

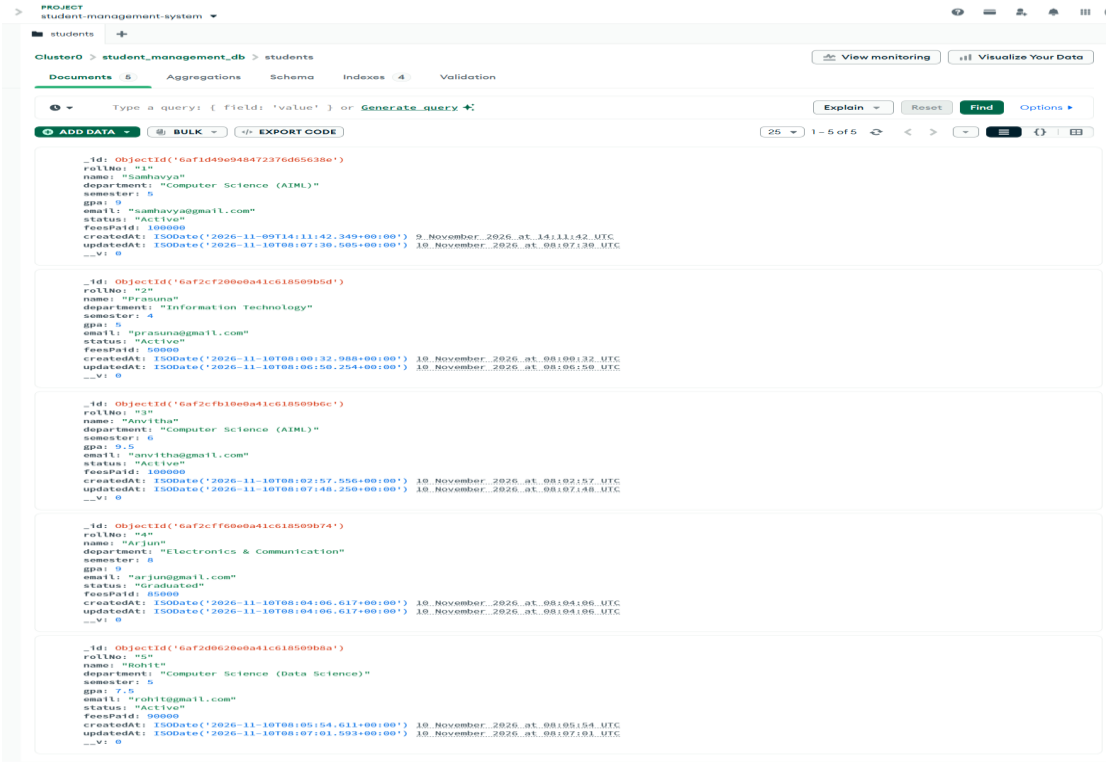


Figure 4. MongoDB Atlas Cloud Collections Browser verifying persisted student entity document

Demonstration and Submission Summary

Deliverable	Submission Link / Target
Source Code Repository	https://github.com/SamhavyaSubhanvitha/fs-lab/tree/main/casestudy
Live Frontend Application	https://student-management-system-8vpsrffnj.vercel.app
Live Backend API Endpoint	https://student-management-system-backend-m17c.onrender.com/api/health
Cloud Database	MongoDB Atlas Managed Cluster (`student_management_db`)
Developer & Roll	Samhavya Subhanvitha (A24126552099)